COS 214

TIVIDY

Generic workflow management system



Group Council of 5

Kiara Ajodhaparsadh u25395344
Cameron Ambraal u25008227
Lesedi Shelile u25110455
Taya Govender u24697274
Ajba Patel u25316584

Research Workflow Management Systems

A **workflow** is a structured sequence of repetitive tasks performed in a particular order to achieve a goal. A **Workflow Management System (WMS)** manages and coordinates these tasks by routing work to the appropriate people or systems and supporting both manual and automated activities. Workflows can be represented using diagrams that show activities, decisions, events, responsibilities and the flow between steps.A **workflow definition** describes the structure and rules of a process, including its steps, variables, dependencies and actions, while a **workflow instance** is a specific execution created from that definition. Activities or work items represent individual units of work and may be performed by users or automatically by the system. Dependencies and events determine when activities can begin and how the workflow progresses, including whether activities can run sequentially or in parallel.Work items are **assigned and routed** according to defined users, roles, conditions or approval rules. Participants and responsibilities can therefore be represented through assigned users or groups, with notifications directing the relevant participant to complete a task. Approval workflows can require sequential or parallel approvals, with requests being **approved, rejected, cancelled or delegated**. Overdue requests can trigger reminders or escalation to other responsible users.Workflow systems also handle **failures and exceptions** through alternative responses, cancellation, delegation or other defined paths rather than simply stopping the entire process. Workflows can interact with **external systems** through automated tasks, APIs and integration tools, allowing information and actions to move between different applications. This enables workflows to coordinate both organisational activities and external software systems.Workflow history is a record the engine builds for each running instance (which activities ran, who acted, when, and with what outcome), and it stays attached to that instance rather than to the definition: an instance migrated to a new definition version keeps its unique identity and its full history audit trail (Camunda, n.d.-b). Monitoring and reporting read this history together with the live status of each instance to show how many cases are running, which are overdue or stuck, and how long each step takes, so they can sit outside the execution logic and change without touching it. A workflow engine is reusable because the process-specific content sits in the definition while the engine only executes common constructs. Workflow patterns describe recurring constructs independently of any specific workflow language, and most languages share the basic constructs of sequence, iteration, splits and joins (van der Aalst et al., 2003), so one engine can run many different processes. This divides the information: the definition describes the activities and their ordering, with business and processing data layered on top of that control flow (van der Aalst et al., 2003), while each instance carries its own identity, progress, audit trail and the definition version it started with (Camunda, n.d.-b). Nested work is modelled in two ways in BPMN: an embedded sub-process sits inside its parent definition, whereas a call activity references a separately defined process, which makes it reusable from many other processes (Camunda, n.d.-a), and the call activity acts as a wrapper that transfers control to the called global process (Flowable, n.d.). The decisions that are configurable rather than hard-coded are the ones that differ between organisations: who a task is assigned to, which approval levels and thresholds apply, deadlines, the conditions on routing decisions, and which sub-process is called, since the called process is resolved at runtime and can be bound to the latest version, the version in the same deployment, or a fixed version (Camunda, n.d.-a). The engine itself only hard-codes the common execution behaviour, such as how a sequence, split or join runs and

what each lifecycle state allows. When a definition changes while work is in progress, running instances keep the version they started with and only new instances use the latest one, so changed definitions can be deployed without disturbing existing cases, and instances of different versions can run in parallel (Camunda, n.d.-b). Where an instance must follow the new definition, a migration plan describes how it moves from one version to another (Camunda, n.d.-b).

References

Camunda. (n.d.-a). Call activity. Camunda 7 Manual.

https://docs.camunda.org/manual/7.7/reference/bpmn20/subprocesses/call-activity/

Camunda. (n.d.-b). Versioning process definitions. Camunda 8 Docs.

https://docs.camunda.io/docs/components/best-practices/operations/versioning-process-definitions/

Flowable. (n.d.). Call activity. Flowable Documentation.

https://documentation.flowable.com/latest/model/bpmn/reference/call-activity

IBM. (n.d.-a). What is a workflow? IBM. https://www.ibm.com/think/topics/workflow

IBM. (n.d.-b). Workflow definitions and workflow instances. IBM Documentation.

https://www.ibm.com/docs/en/openpages/9.0.0?topic=fundamentals-workflow-definitions-workflow-instances

IBM. (n.d.-c). Workflow elements. IBM Documentation.

https://www.ibm.com/docs/en/sig-and-i/11.0.2?topic=planning-workflow-elements

Microsoft. (2025, October 16). Approve or reject documents in workflows. Microsoft Learn.

https://learn.microsoft.com/en-us/dynamics365/business-central/across-how-use-approval-workflows

ProjectManagement.com. (2019, November 29). Work flow diagramming.

https://www.projectmanagement.com/wikis/233095/Work-Flow-Diagramming

van der Aalst, W. M. P., ter Hofstede, A. H. M., Kiepuszewski, B., & Barros, A. P. (2003).

Workflow patterns. Distributed and Parallel Databases, 14(1), 5-51.

Define the Scenarios and System

Scenario 1 : A general hospital system

The workflow follows a patient's admission to the hospital, through either emergency or general admission, all the way to initial treatment.

Environment: The hospital receives patients in two main ways: emergency admission, where paramedics bring the patient in, or general admission, where the patient walks in and is registered by the secretary. This scenario covers both paths. The emergency path starts when the paramedics pick up the patient. The general path starts when the patient arrives at reception. The two paths have different early stages but then join. Once a patient has been taken in by the nursing staff, both paths share the same doctor acceptance, assessment, room allocation and cleaning stages, until the patient is assessed in a room, ICU or surgery, or is discharged.

Participants and roles: The main participants focused on include:

- **Paramedics (emergency only):** pick up the patient, transport them and update the transport status.
- **Secretary**: registers incoming emergency patients and walk-in patients, and notifies the nursing staff.
- **Nursing staff**: triage walk-in patients, receive emergency patients, prepare patients, and coordinate and communicate with the doctor.
- **Doctor**: accepts the cases, assesses the patient and decides on the room.
- **Cleaning** staff: clean the area after the patient leaves.

Major pieces of work:

Emergency admission path:
1. Paramedic pickup and transportation
2. Patient registration on arrival

3. Room or bay preparation (in parallel with transportation)
4. Patient handover from paramedic to nurse
5. Nurse intake and preparation

General admission path:
1. Patient arrival and registration at reception
2. Nurse triage (vital signs taken and priority level assigned)
3. Waiting queue, ordered by triage priority
4. Nurse intake and preparation

Shared stages (both paths):
1. Doctor case acceptance
2. Doctor assessment and treatment
3. Room allocation by the doctor (admit to a general room, ICU or surgery, or discharge)
4. Room departure cleaning

Decisions and routing points: Deciding on the admission route (emergency or general). Assigning a triage priority level to walk-in patients. Whether a walk-in patient's condition is critical enough to be moved onto the emergency path. Picking the room type the patient should be routed to. Whether the doctor accepts the case or escalates the patient to a specialist. Whether the patient is discharged, admitted, or transferred to ICU or surgery.

Approval:
- **Hospital admission approval:** the hospital formally accepts the patient (both paths).
- **Doctor case acceptance:** needed before treatment begins. If it isn't accepted within a deadline, the case is passed on to another doctor. For walk-in patients, the deadline depends on the triage priority, so more urgent patients have shorter deadlines.
- **Room allocation approval:** ICU and surgery transfers require approval before the patient is moved.

Dependencies:
- Cleaning only begins after the patient is no longer in that area.
- Nurse intake can't begin until handover (emergency) or triage (general) is complete.
- Triage can't begin until the walk-in patient has been registered.
- Walk-in patients are seen in order of triage priority, not arrival time.
- Treatment can't begin until the doctor has accepted the case.

Failure and exception paths:

- **Paramedic transport fails or is delayed:** the hospital is informed, and preparation is paused or cancelled.
- **Triage finds a critical condition:** the walk-in patient is moved straight onto the emergency path.
- **A walk-in patient waits longer than their priority allows:** their priority is escalated.
- **A walk-in patient leaves before being seen:** the workflow ends in a “left without being seen” outcome.
- **The doctor doesn’t respond within the deadline:** the case is rerouted to another doctor.
- **The patient’s health deteriorates during intake:** escalated to emergency treatment.
- **The patient doesn’t survive:** the workflow ends in a deceased-patient outcome.

External systems:

- Ambulance dispatch system (emergency path): paramedic status updates (picked up, en route, arrived).
- Hospital patient records (both paths): the patient’s history is retrieved on arrival or registration and updated during treatment.

Expected outcome:

The patient is admitted to a ward, transferred to ICU or surgery, discharged, recorded as deceased, or, for walk-ins only, recorded as having left without being seen. The full history is recorded and archived for future reference.

Scenario 2 : A Law Firm System

The workflow follows a new client's case from the first inquiry until the firm formally opens, declines or refers the matter.

Environment: The law firm receives prospective clients in two main ways: new clients (walk-ins, phone or online inquiries, and referrals) or existing clients opening a new matter. This scenario mainly focuses on the new-client intake path, starting when the inquiry is received and ending when the matter is opened and assigned to a legal team, declined, or referred to another firm. Existing-client matters exist in the environment but are treated as a separate process that would reuse the same generic TIVIDY mechanisms.

Participants and roles: The main participants focused on include:

- **Intake coordinator (receptionist):** receives the inquiry, captures the client's details and the type of legal problem, and notifies the right practice group.
- **Paralegal:** collects and organises the client's documents, verifies the client's identity and prepares the case file.
- **Compliance officer:** runs the conflict-of-interest check and clears or blocks the matter.
- **Associate attorney:** evaluates the case, decides whether the firm can take it, and drafts the fee estimate and engagement letter.
- **Partner:** gives final approval for accepting the case, especially high-value or high-risk matters and non-standard fee arrangements.
- **Billing clerk:** sets up the client account, issues the retainer invoice and confirms Payment.

Major pieces of work:

1. Inquiry received and logged
2. Client details and identity verification
3. Conflict-of-interest check (in parallel with document collection)
4. Collection of the client's supporting documents
5. Preliminary case evaluation by the associate
6. Case acceptance approval
7. Fee estimate and engagement letter
8. Retainer invoice and payment
9. Matter opened and assigned to a legal team (or a non-engagement letter sent)

Decisions and routing points: Deciding the intake route: urgent, with a court date or legal deadline close, or standard. Routing the case to the correct practice area (criminal, family, commercial or litigation). Whether a conflict is found, which leads to decline or to requesting a waiver. Whether the associate accepts the case or escalates it to a more senior attorney. Whether the case is accepted, declined, or referred to another firm.

Approval:

- **Conflict clearance approval:** the compliance officer must clear the conflict check before anyone evaluates the case.
- **Case acceptance approval**: this goes up a chain of authority, from associate to senior associate to partner, depending on the value and risk of the case. If a level doesn't decide within its deadline, the case is passed on to the next level.
- **Fee arrangement approval:** non-standard fees, such as contingency fees or discounted rates, need partner approval before the engagement letter is sent.

Dependencies:

- Case evaluation can't begin until the conflict check is cleared.
- The engagement letter can't be sent until the case has been accepted.
- The matter can't be opened until the client has signed the engagement letter and paid the retainer.
- The conflict check and document collection run in parallel, but both must finish before evaluation.

Failure and exception paths:

- **Conflict of interest found:** the case is declined, or a waiver is requested from the affected parties. If no waiver is given, the workflow ends in a declined outcome.
- **Client's identity can't be verified:** intake is stopped and the client is informed.
- **Client doesn't provide documents within the deadline:** a reminder is sent. If they still aren't received, the intake is closed as withdrawn.
- **An attorney doesn't respond within the deadline:** the approval is escalated to the next level of authority.
- **A legal time limit is about to expire:** the case is marked urgent and escalated straight to a partner.
- **Client doesn't sign or doesn't pay the retainer:** the intake ends in a withdrawn outcome.

External systems:

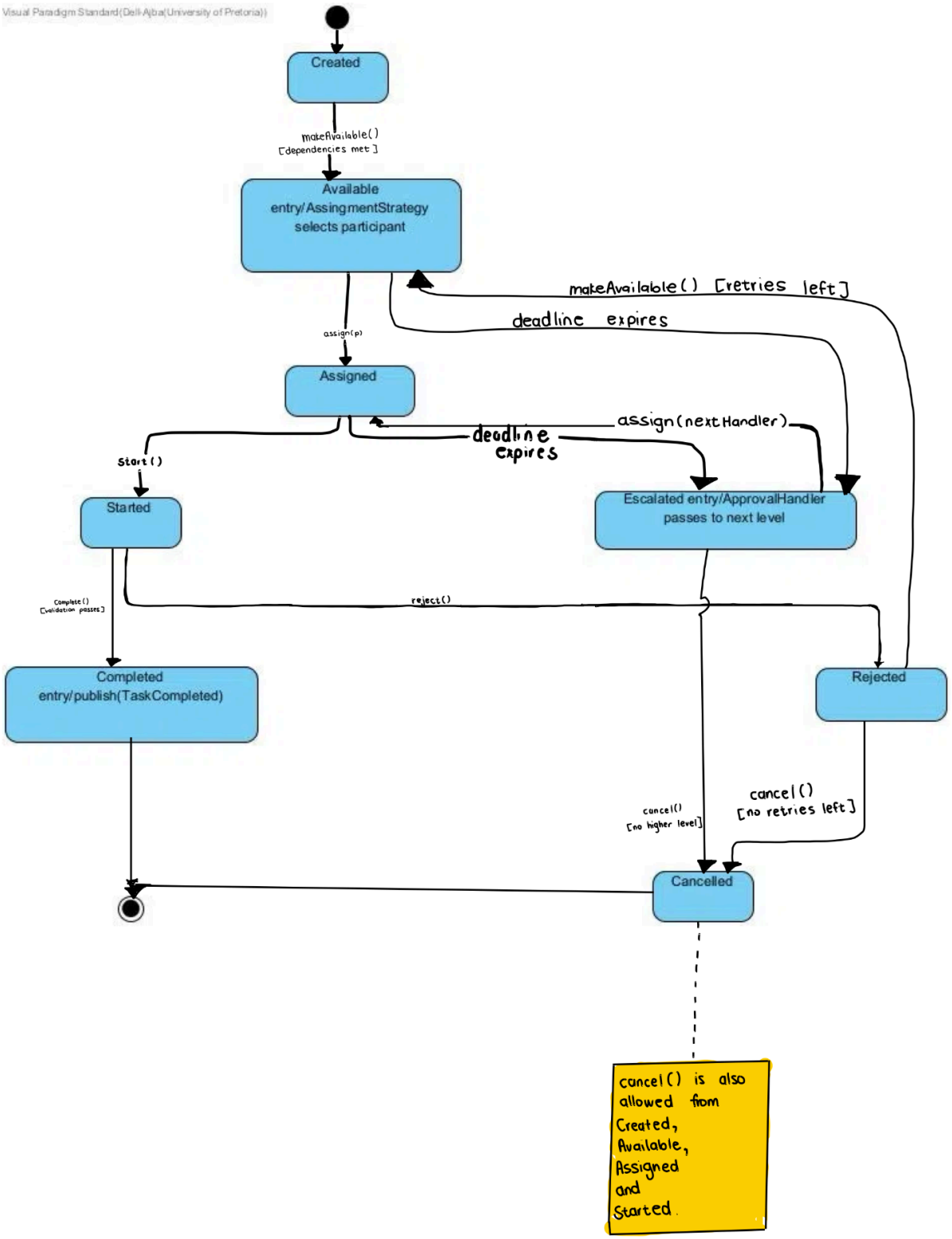
- **Conflict-check database:** the firm's records of past and current clients and opposing parties, searched during the conflict check.
- **Practice management and billing system:** opens the client account and matter number, issues the retainer invoice and confirms payment.

Expected outcome:

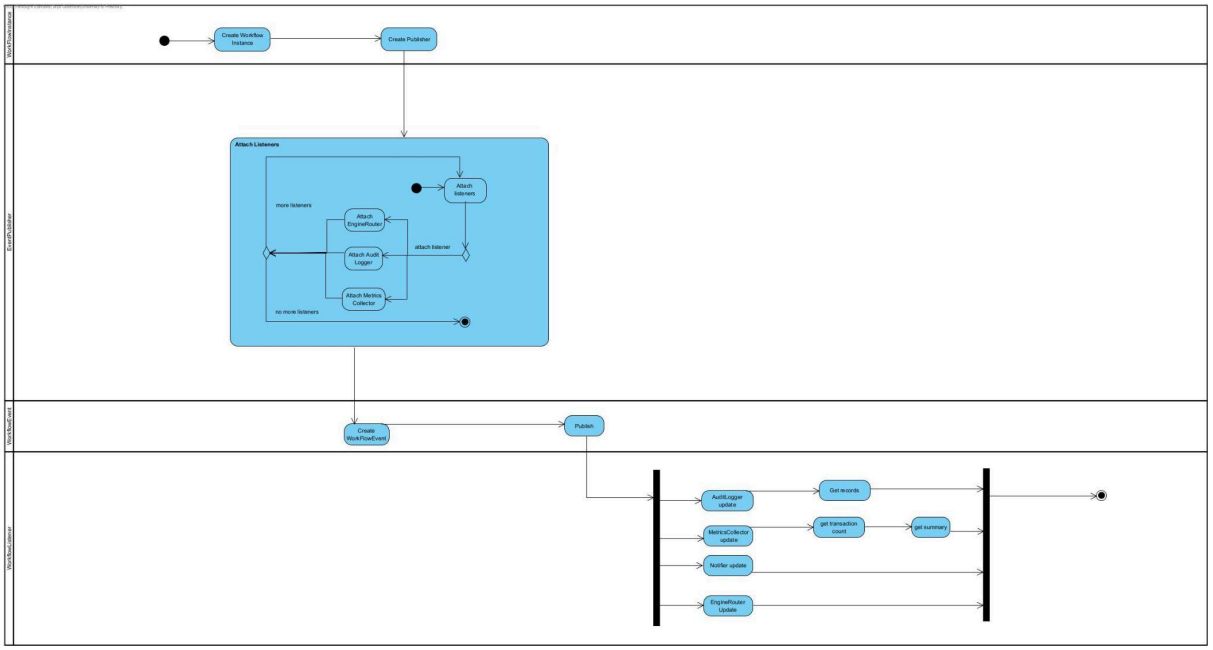
The matter is opened and assigned to a legal team, declined with a non-engagement letter, referred to another firm, or closed as withdrawn. The full history is recorded and archived for compliance and future reference.

UML Activity Diagrams

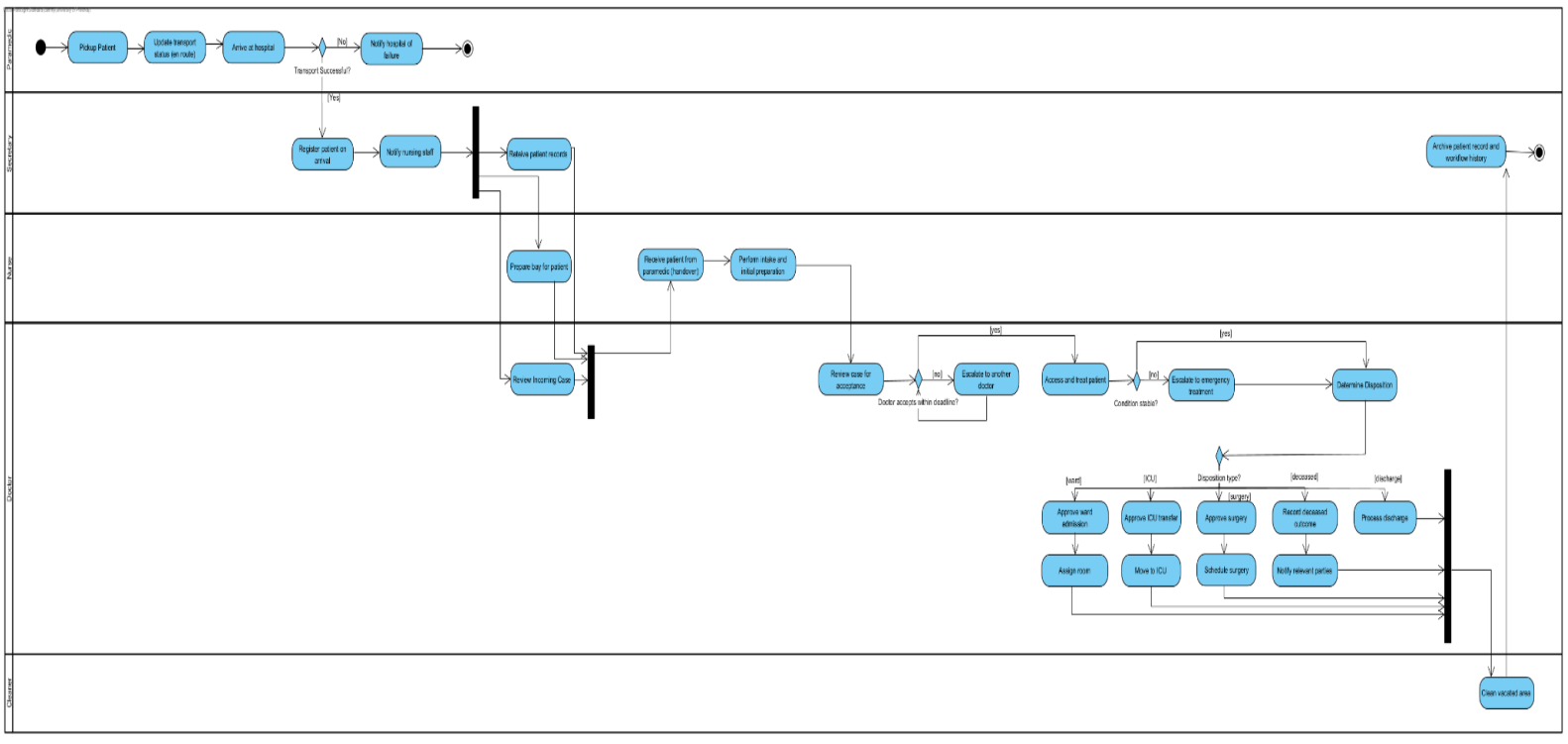
Activity Diagram 1:



Activity Diagram 2:



Activity Diagram 3:



Design Patterns

DESIGN RATIONALE

TIVIDY is a generic workflow management system. It stores workflow definitions, creates independent running instances from them, routes work to participants, enforces approvals, reacts to events and records what happened. Our emergency admission scenario exercises the engine but does not shape it. Only `EmergencyAdmissionDirector`, the two adapters and their external APIs mention the hospital.

Definition versus instance. A `WorkflowDefinition` is frozen once built and owns the root of a `WorkItem` tree. `createInstance()` clones that tree, so each `WorkflowInstance` has its own state, assignees and data, and changing one cannot affect another.

Nested work. Work is a Composite: `Task` is the leaf, and `SequentialStage` and `ParallelStage` are composites. The engine treats every `WorkItem` the same way, so no type checks are needed. A Builder assembles the tree stage by stage, and an Iterator finds the next available work without exposing how stages store children.

Changing behaviour. State controls a task's lifecycle and rejects invalid operations. Assignment is a Strategy chosen from outside, whether by role, workload or rule. Approval and escalation use a Chain of Responsibility, and Decorators add validation, auditing and priority without creating new kinds of task.

Reacting to events. `EventPublisher` notifies listeners of `WorkflowEvent` objects. Routing, audit, notification and metrics are separate listeners, so new interested parties can be added without changing the core. We used `MetricsCollector` instead of Visitor for monitoring because it reuses this mechanism.

History and recovery. Actions are Command objects with `undo()`, and each keeps a Memento of the work item taken beforehand.

External systems. The engine knows only `ExternalConnector`. Two Adapters translate it to each external system's interface.

`TIVIDYFacade` gives clients a single simple interface to the whole engine.

1. COMPOSITE

Problem solved:
Work is nested (stages, sub-processes, individual tasks). The engine must start, complete and report on any piece of work without knowing whether it is on task or a whole stage. We are able to uniformly perform operations on objects(tasks) and groups of objects(whole stage).

How do the participants collaborate?
`WorkflowInstance` holds a root `WorkItem`. When the engine calls `complete()` or `isFinished()` on a `Stage`, it forwards the call to its children and aggregates the results (a stage is finished when all children are).

What varies?
The shape and depth of the work hierarchy. Each definition has a different tree.

Appropriateness
Emergency admission, patient discharge and treatment all contain stages with tasks inside them. Without Composite, the engine would need separate code paths for a singular task and a group, which is the type-checking the brief forbids.

Participants

Component	WorkItem
Leaf	Task
Composite	Stage (SequentialStage and ParallelStage)

2. BUILDER

Problem solved:
A workflow definition is complex - nested stages, tasks, strategies, approval chain, decorators and difficult to construct in one constructor.

How do the participants collaborate and what varies?
The director calls the builder's steps in order, then retrieves the finished definition. The engine later creates independent `WorkflowInstances` from it, so changing one instance doesn't change the definition or other instances.
The content of the definition, and the representation it is built into.

Appropriateness
It separates constructing a definition from running it, which is the brief's definition-versus-instance split.

Participants

Builder	WorkflowBuilder
ConcreteBuilder	DefinitionBuilder
Director	EmergenyAdmissionDirector
Product	WorkflowDefinition

3. ITERATOR

Problem solved:
The engine, reports and the UI need to walk the work tree without knowing how `Stage` stores its children.

How do the participants collaborate and what varies?
`TIVIDYFacade` asks the root `WorkItem` for an iterator to find available work. A `DepthFirstIterator` can also be used when the system needs to inspect the complete workflow tree. The traversal order and filtering behaviour can vary without changing the structure of `Stage`.

Appropriateness
After one task completes, the engine must find what became available, and different views need different traversals. Iterator keeps that logic out of `Stage` and hides its storage.

Participants

Aggregate	WorkItem
Iterator	WorkItemIterator
ConcreteIterators	DepthFirstIterator, AvailableItemsIterator

4. STATE

Problem solved:
A work item's valid operations depend on where it is in its lifecycle. You can't start something that is only created, or complete something that was cancelled.

How do the participants collaborate and what varies?
`Task` delegates `assign()`, `start()`, `complete()`, `reject()` and `cancel()` to its current `TaskState`. The state either performs the operation and sets the next state, or rejects it as invalid.

Appropriateness
Without State you'd have `if (status == ...)` chains in every `Task` method. Here the lifecycle is real behaviour that changes the object's responses.

Participants

Context	Task
State	TaskState
ConcreteStates	CreatedState, AvailableState, AssignedState, StartedState, CompletedState, RejectedState, CancelledState, EscalatedState

5. STRATEGY - Assignment

Problem solved:
Different organisations assign the next task by role, by workload, or by a rule calculated from workflow data, and `Task` must not change when this does.

How do the participants collaborate and what varies?
When a task becomes available, it calls `strategy.selectParticipant(task, participants)` and then assigns the result. The assignment algorithm, chosen per task or per definition.

Appropriateness:
The brief names this exact variation. The strategy is chosen from outside by the definition, which is why it is Strategy and not State.

Participants

Context	Task
Strategy	AssignmentStrategy
ConcreteStrategy	RoleBasedAssignment, WorkloadAssignment, RuleBasedAssignment

6. CHAIN OF RESPONSIBILITY

   Problem solved:
   Approval can involve several levels, and a request must move up to the next level if someone rejects the authority question or doesn't respond in time. This will be specifically responsible for approval authority and escalation.

   How do the participants collaborate and what varies?
   The request enters at the first handler. If the handler can decide, it does. If the request requires a higher authority than its role, or its deadline expires, it passes the request to `next`. The scenario's configuration builds the chain and adds no scenario classes to the core. The number of levels, the thresholds and roles, and the escalation order vary.

   Appropriateness:
   This pattern will pass it up until someone with enough authority handles it. Adding a level means adding one more link, with no change to existing handlers.

   Participants

Handler	ApprovalHandler
ConcreteHandler	ApprovalLevel
Request	ApprovalRequest (carries the required authority level - ICU or surgery transfer)

7. OBSERVER - used in both events and metrics

   Problem solved:
   Completing an activity or receiving an external event must trigger several independent reactions (make the next item available, write audit history, notify people, update metrics) without the source knowing about any of them.

How do the participants collaborate and what varies?
A state change in a `Task` publishes a `WorkflowEvent`. The publisher notifies every registered listener. `MetricsCollector` is just one more listener that counts transitions and records time spent in each state, which covers monitoring.
Which parties care about events and what each does with them vary.

Appropriateness:
Adding a new interested party means registering a new listener, with no change to the workflow core.

Participants

Subject	EventPublisher
Observer	WorkflowListener
ConcreteObservers	EngineRouter, AuditLogger, Notifier, MetricsCollector

8. COMMAND

Problem solved:
Actions on work (assign, start, complete, reject) must be recorded in history and sometimes undone.

How do the participants collaborate and what varies?
A participant action creates a command. The invoker executes it, then pushes it onto the history. Undo pops it and calls `undo()`.
The set of actions, and what each one does and reverses vary.

Appropriateness:
The spec requires retained actions and undo. Objects that represent actions give you an audit trail and undo without extra logic inside `Task`.

Participants

Command	WorkflowCommand
Receiver	Task

Invoker	CommandHistory
Concrete Commands	AssignCommand, CompleteCommand, RejectCommand

9. MEMENTO

Problem solved:
Undoing some actions is easier and safer by restoring a saved state than by writing an inverse operation, e.g. rolling a workflow back after a failure.

How do the participants collaborate and what varies?
Before `execute()`, a command asks the task for a snapshot and stores it. On `undo()`, the command hands the snapshot back and the task restores itself. The task's internals stay private.
What state needs saving, and how far back rollback goes vary.

Appropriateness:
Failure recovery needs reliable restoration. Memento keeps snapshots encapsulated and works together with Command.

Participants

Originator	WorkItem
Memento	TaskMemento
Caretaker	CommandHistory

10. DECORATOR

Problem solved:
Tasks sometimes need extra behaviour without becoming a different kind of task.

How do the participants collaborate and what varies?
A decorator wraps a task and adds its behaviour before or after forwarding the call, for example `ValidatedTask.complete()` checks required data, then delegates. Decorators can be stacked.
The combination of extra behaviours attached to each task, chosen when the workflow is built vary.

Appropriateness:
Without Decorator we would need subclasses for every combination for example `ValidatedPriorityAuditedTask`.

Component	WorkItem
ConcreteComponent	Task
Decorator	TaskDecorator
ConcreteDecorators	ValidatedTask, AuditedTask, PriorityTask

11. ADAPTER

Problem solved:
TIVIDY must call external systems whose interfaces weren't designed for it.

How do the participants collaborate and what varies?
The task calls `connector.send(...)`. The adapter translates this into the external system's own calls and translates the response back.
The external system and its API vary. TIVIDY remains dependent only on the stable ExternalConnector Target interface, while each Adapter translates that interface into the corresponding external API.

Appropriateness:
The generic engine only knows `ExternalConnector`, and the scenario supplies the adapters. This is how the scenario avoids leaking into the core.

Participants

Target	ExternalConnector
Adaptees	PatientRecordsAPI, AmbulanceDispatchAPI
Adapters	PatientRecordsAdapter, AmbulanceAdapter
Client	ExternalTask - a task that calls an external connector

12. PROTOTYPE

Problem solved:
A `WorkflowDefinition` is a reusable template, but every running case needs its own tree with its own progress, assignees and data. If instances shared the definition's tree, completing a task in one claim would change every other claim. Building the tree again with the Builder for each new instance would also be wasteful and would tie instance creation to the Builder.

How do the participants collaborate ?

- The Builder produces a `WorkflowDefinition` holding a root `Stage`, and the definition is then frozen.
- `TIVIDYFacade` asks the definition for an instance, and `createInstance()` calls `clone()` on the root.
- Each `Stage` clones its children recursively, so the whole tree is duplicated.
- The new `WorkflowInstance` takes ownership of the cloned tree and records the definition's `version`.

What varies ?

The shape of the tree being copied, and which concrete work-item types it contains. `createInstance()` never needs to know either, because each node knows how to clone itself.

Appropriateness:
Prototype is appropriate because TIVIDY must create multiple independent workflow instances from the same reusable workflow definition. Cloning the workflow tree prevents instances from sharing execution state while allowing the definition to remain unchanged.

Participants

Prototype	WorkItem
Concrete prototypes	Task, SequentialStage, ParallelStage, TaskDecorator
Client	WorkflowDefinition

13. FACADE

Problem solved:

TIVIDY contains several interacting subsystems, such as workflow definitions, workflow instances, task execution, assignment, commands and event handling. A client should not need to understand or directly coordinate all of these subsystems just to perform common workflow operations. .

How do the participants collaborate and what varies?
The client communicates with `TIVIDYFacade` instead of directly accessing the individual workflow subsystems. The facade coordinates the required subsystem objects to perform operations such as creating a workflow instance, starting a workflow, assigning work and completing a task. The internal subsystem structure and the sequence of calls required to perform an operation can change without requiring changes to the client. The internal TIVIDY subsystems and how they collaborate to perform high-level workflow operations varies. The client-facing interface remains simple and stable.

Appropriateness:
TIVIDY contains many design patterns and therefore several collaborating objects. Without a facade, application code would need to know about and coordinate objects such as `WorkflowDefinition`, `WorkflowInstance`, `CommandInvoker`, `Task` and `EventPublisher`. `TIVIDYFacade` provides a single simplified entry point while keeping the internal architecture encapsulated. This is appropriate because TIVIDY is intended to be a reusable workflow management system that may have different client applications.

Participants

Facade	TIVIDYFacade
Subsystem classes	WorkflowDefinition, WorkflowInstance, CommandHistory, Task, EventPublisher, ApprovalHandler
Client	main

14. PROXY

Problem solved:
Clients interacting with TIVIDY workflow services need security checks and access control - such as verifying user permissions and task ownership, before sensitive workflow operations, like starting instances or assigning tasks are executed.

Without a Proxy, this authentication logic would be tightly coupled with the core workflow execution logic or duplicated across client applications.

How do the participants collaborate and what varies?

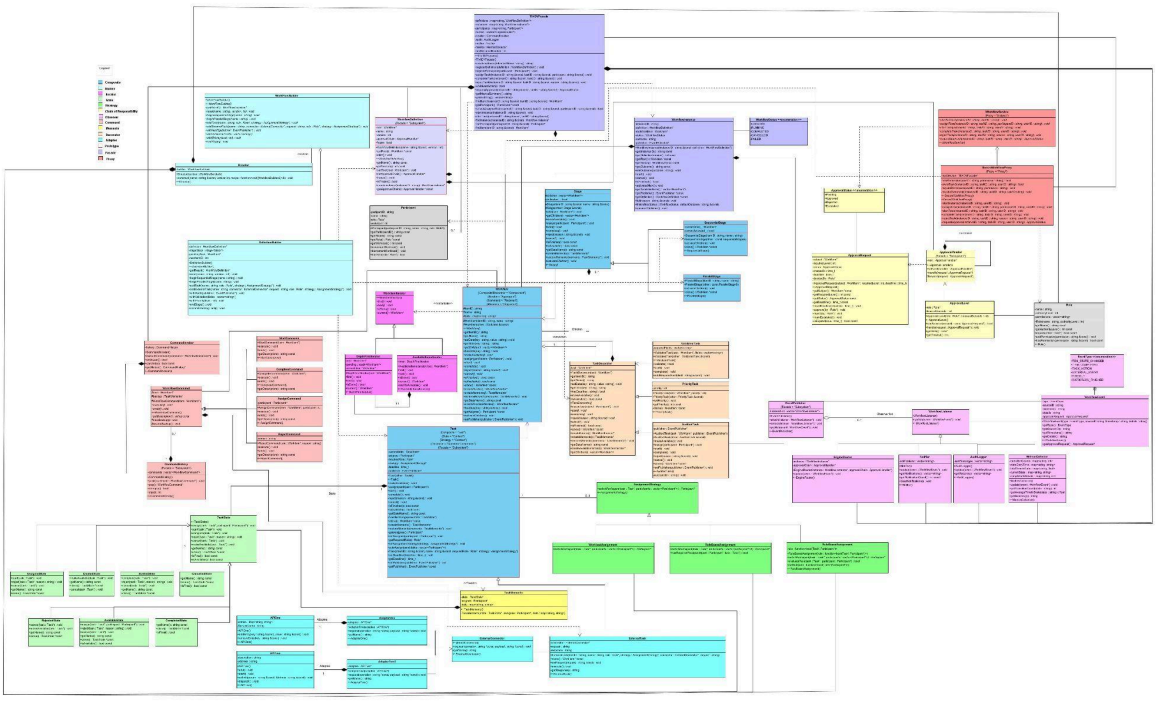
The client communicates with `SecureWorkflowProxy` instead of directly accessing the `TIVIDYFacade`, both sharing the common `IWorkflowService` interface. The proxy intercepts client requests, performs security validations by using internal methods like permission checks and ownership verification. These methods, if authorized, delegates the call to the underlying `TIVIDYFacade` real service. What varies are the security rules, access control policies, or additional proxy behaviours, while the underlying workflow execution logic and the client interface remain completely independent and stable.

Appropriateness

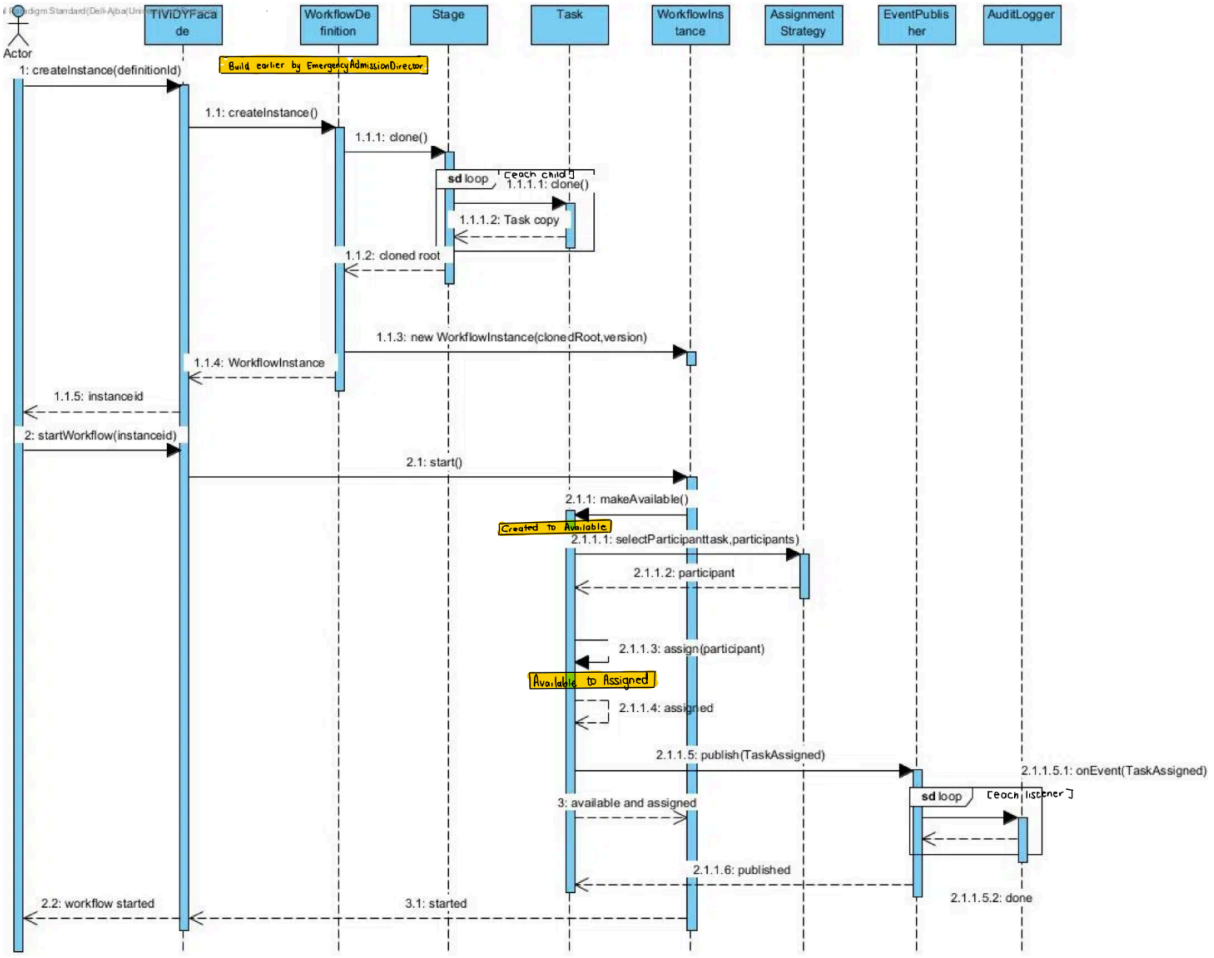
This is appropriate because TIVIDY is a reusable workflow management system where different users have distinct roles and permissions that must be enforced securely. The Proxy Pattern allows access control to be introduced transparently without altering the core `TIVIDYFacade` or client code. It maintains a clean separation of concerns by isolating security management within `SecureWorkflowProxy` while presenting a uniform interface to the client.

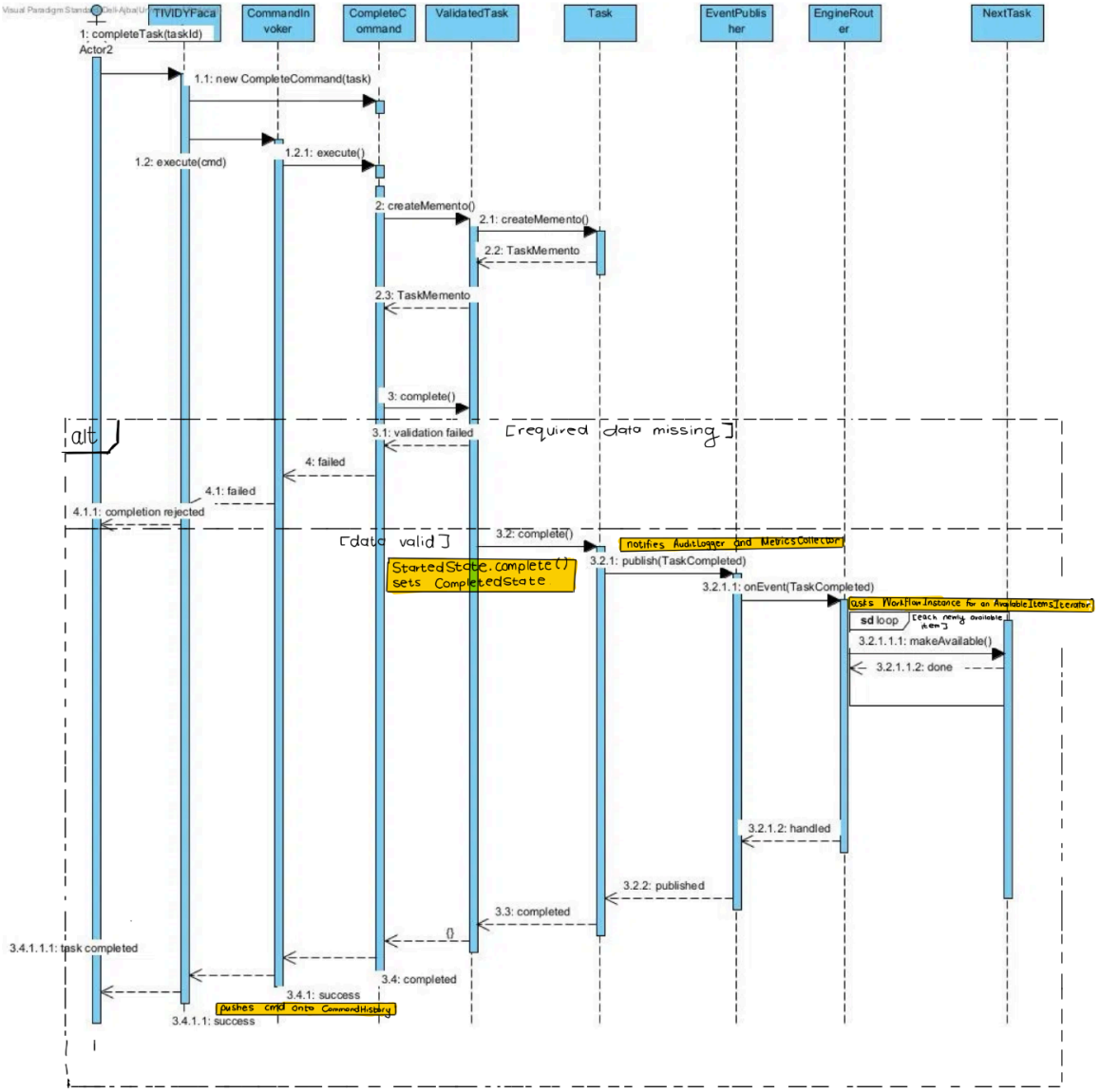
Proxy	SecureWorkflowProxy
Subject	IWorkflowService
Client	TIVIDYFacade

UML Class Diagram



Runtime Behaviour Diagrams





Design Decisions and Revisions

Revision 1: The product of the Builder

Problem. We needed to decide what the Builder constructs and which class is responsible for turning that product into running workflows.

Original design. `WorkflowDefinition` was the product of the Builder. The user set its attributes through the Builder, which then created a `WorkItem`.

First revision. We made `WorkItem` the product instead, with the Builder creating work items directly. This removed the definition from the Builder's role.

Second revision. Once the Facade was added to the system, `WorkflowDefinition` became the product again. It now holds the root of the `WorkItem` composite tree and is frozen once built. It also creates `WorkflowInstance` objects, and each instance receives its own cloned copy of the tree, which allows many instances of the same workflow to run side by side without affecting each other. We added the Prototype pattern for this cloning.

Reason. The Builder returning a bare `WorkItem` gave the Facade no single object to register, store by name and version, and create instances from. Making `WorkflowDefinition` the product again separates the description of a process from a running process, as the brief requires. Cloning also guarantees that completing a task in one patient's admission cannot change another's. A versioned definition also preserves the meaning of older workflows when a new version is created.

Diagrams updated: `WorkflowDefinition` as the Builder's product, composition to the root `WorkItem`, `clone()` on the Composite and Decorator classes, the Builder and Prototype sections of the pattern documentation, and the sequence diagram for creating and starting a workflow instance.

Revision 2: Participants as a class

Problem. Several classes need to refer to a participant: a worker or employee who is assigned work or approves requests. We first wanted to avoid adding extra classes.

Original design. Participant parameters were plain strings, such as a name or an ID.

Change. We added a `Participant` class holding an ID, a name, a `Role` and a current workload. `Role` is a small class with a name and an authority level. Assignment, approval, commands and the Facade now work with `Participant` objects, and the hospital roles are created as data by the director rather than as subclasses.

Reason. A string cannot carry the information the design needs. `WorkloadAssignment` has to compare participants by workload, `RoleBasedAssignment` has to match their role to a task, and `ApprovalLevel` uses the authority level to decide who may approve a request. The Facade also needs to register participants and look them up by ID. The two small classes give all of this meaningful information in one place without adding hospital names to the core.

Revision 3: Replacing Visitor with Observer for monitoring

Problem. TIVIDY must produce monitoring and reporting information without placing every report operation inside the core work-item classes.

Original design. We first planned a Visitor over the work-item tree, then considered a `ReportStrategy` hierarchy that would walk the tree with an Iterator.

Change. We removed both. `MetricsCollector` is now one more `WorkflowListener` in the Observer pattern. It records transition counts and the time work items spend in each state, and produces a summary on request.

Reason. Visitor depends on a stable class hierarchy, and ours was still changing. A report strategy would have added a second hierarchy for a small benefit. The Observer mechanism already existed for routing, audit and notifications, so monitoring reuses it by adding one class. The design still has 13 patterns, which is inside the required range.

Diagrams updated. The class diagram (Visitor and report classes removed, `MetricsCollector` added to the Observer group) and the pattern justifications.

Revision 4: Lifecycle operations moved up to `WorkItem`

Problem. The engine, commands and decorators must act on a work item without knowing whether it is a single task or a whole stage. With the lifecycle operations on `Task` only, the engine would need to downcast, which the brief forbids.

Original design. `assign()`, `start()` and `complete()` were declared on `Task`, and the Command classes took a `Task` as their receiver.

Change. These operations are now declared on `WorkItem`. `Stage` implements them by acting on its children, and `add()` stays on `Stage` only. Commands now target a `WorkItem` and each command owns the memento it takes before executing.

Reason. Tasks and stages are treated uniformly, which is the purpose of Composite. It also fixed a defect: `TaskDecorator` extends `WorkItem`, not `Task`, so commands aimed at `Task` could not act on decorated tasks. Undo now works for decorated and undecorated items alike.

Diagrams updated. The class diagram (commands linked to `WorkItem`, composition from `WorkflowCommand` to `TaskMemento`), the Command and Memento sections, and the sequence diagram for completing a work item.

Revision 5: Keeping the facade generic

Problem. The facade depended on `EmergencyAdmissionDirector`, which tied the generic engine to the hospital scenario. The brief says the scenario must not determine the architecture of the workflow manager.

Original design. `TIVIDYFacade` had one operation, `createWorkflow()`, and a dependency on the director.

Change. The dependency was removed. The facade now offers only generic operations: registering definitions and participants, creating and starting instances, assigning, completing and rejecting tasks, undoing the last action, requesting approval, and retrieving metrics and the audit log. The scenario code calls the facade, never the other way round. The hospital-specific classes are limited to the director, the two adapters and their external APIs.

Reason. Nothing in the core mentions patients or ambulances, so the engine could run other processes such as procurement or onboarding without modification.

Diagrams updated. The class diagram (director dependency removed, facade operations added) and the scenario description

Ongoing history

We will keep recording design changes in this format as the project continues, including any that result from tutor feedback or from problems found during implementation.

Team Contribution Statement

We split up the work by each team member working on different tasks.

Kiara worked on Task 4 and contributed to task 5 and 7. Taya worked on Task 5, 7 and did an activity diagram as part of task 3. Lesedi contributed to the research and did activity diagram 1. Cameron handled task 2 and did activity diagram 3. Ajba contributed to the Task 1 research and did the diagrams for Task 6.

We collaborated on Visual Paradiam to share versions of the UML and compute all diagrams. We also created a google doc so we would be able to work live and simultaneously share our writeups. We created our gitHub repository that will be used in the future to share our code.

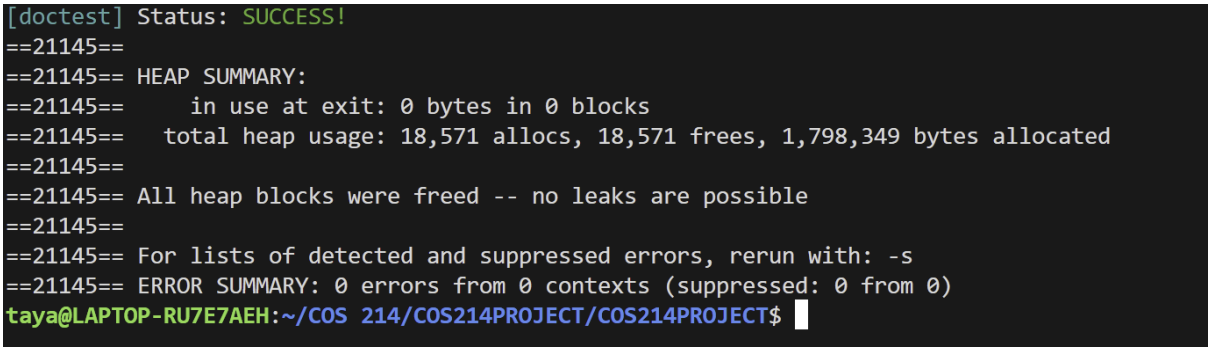
GitHub Workflow

Repository Link

https://github.com/LesediShelile/COS214PROJECT.git

GDB and Valgrind evidence

Valgrind was used during our testing phase along with Doctest.



GDB was also used to ensure there was no segmentation faults in the code as well as bug detection.

